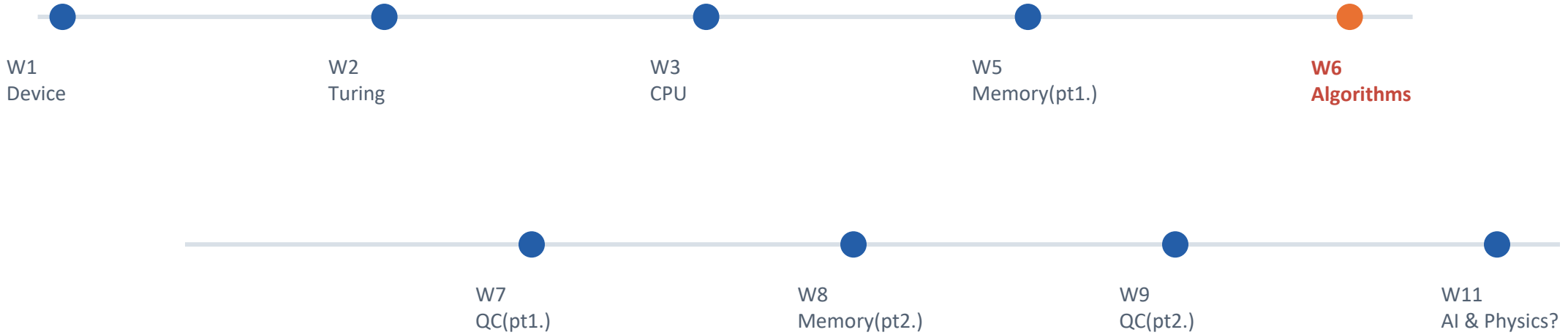
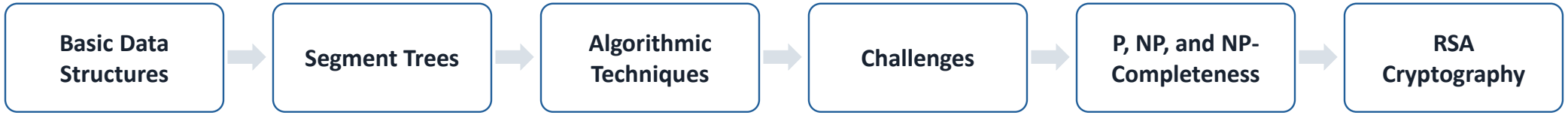
From Efficient Computation to Computational Hardness

Jaehyun Choi
Department of Computer Science and Engineering
Seoul National University

RoadMap

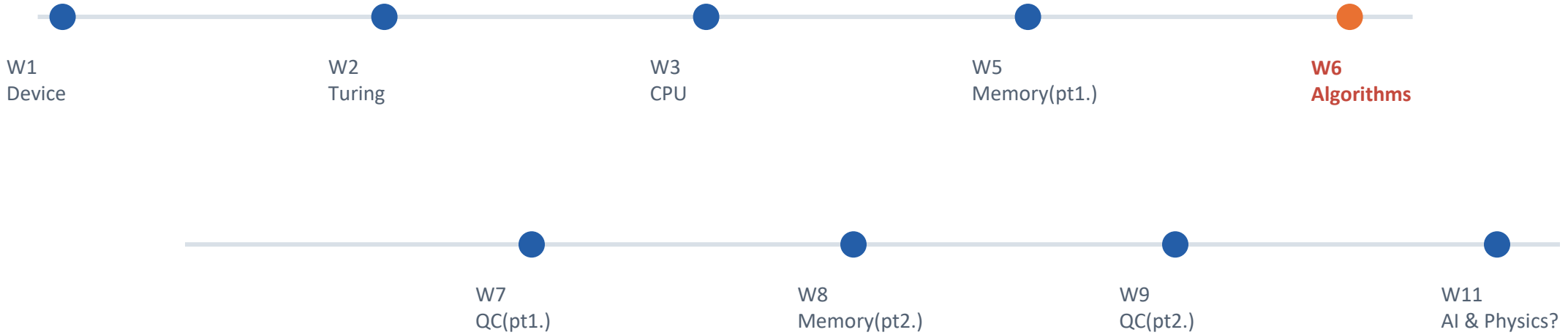

W1
Device
W2
Turing
W3
CPU
W5
Memory(pt1.)
W6
Algorithms
W7
QC(pt1.)
W8
Memory(pt2.)
W9
QC(pt2.)
W11
AI & Physics?

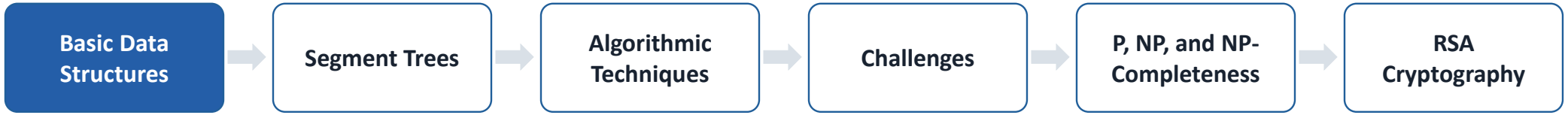

Basic Data Structures
Segment Trees
Algorithmic Techniques
Challenges
P, NP, and NP-Completeness
RSA Cryptography

Key Question

- How can data structures make repeated operations more efficient?
- When should we use greedy algorithms or dynamic programming?
- How can a segment tree be used to optimize a dynamic programming solution?
- What makes some computational problems fundamentally harder than others?
- How can computational difficulty be used in cryptography?

RoadMap


W1
Device
W2
Turing
W3
CPU
W5
Memory(pt1.)
W6
Algorithms
W7
QC(pt1.)
W8
Memory(pt2.)
W9
QC(pt2.)
W11
AI & Physics?


Basic Data Structures
Segment Trees
Algorithmic Techniques
Challenges
P, NP, and NP-Completeness
RSA Cryptography

Why Efficiency Matters

- Suppose a computer performs 10^8 operations per second.

Complexity	$n = 10^3$	$n = 10^5$	$n = 10^6$
$O(\log n)$	Fast	Fast	Fast
$O(n)$	Fast	Fast	Fast
$O(n \log n)$	Fast	Fast	Usually feasible
$O(n^2)$	Fast	Slow	Impossible
$O(2^n)$	Impossible	Impossible	Impossible

- A correct algorithm may still be unusable.

What Is a Data Structure?

- A data structure defines
  - How data is organized
  - Which operations are efficient
  - What information can be accessed quickly
- Different problems require different operations
  - Problem requirements → Data structure choice → Running time

Stack

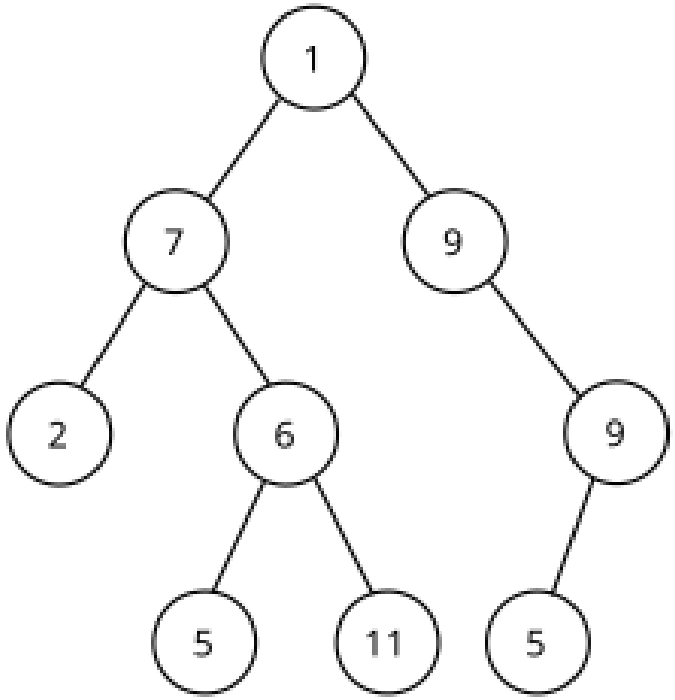
- A stack follows
  - LIFO : Last In First Out
- Main operations
  - push(x) — insert an element
  - pop() — remove the most recent element
  - top() — inspect the most recent element
- All operations : $O(1)$

Queue

- A queue follows
  - FIFO : First In First Out
- Main operations
  - push(x) - insert an element
  - pop() - remove the element that was inserted first
  - front() — inspect the element that was inserted first
- All operations : $O(1)$

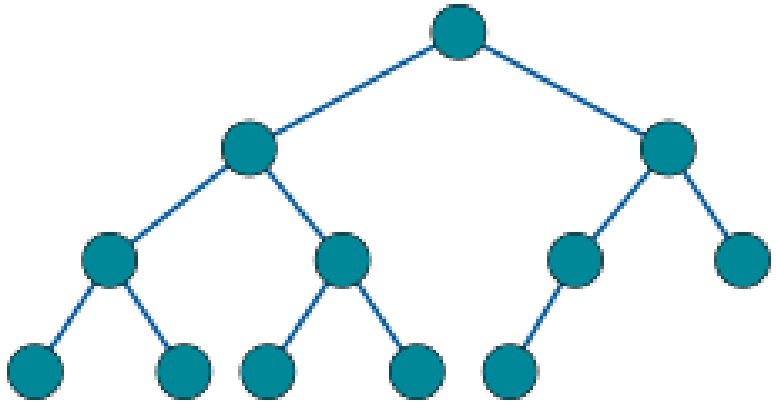
Binary Tree

- A binary tree is a tree in which each node has at most two children
  - Left child
  - Right child
  - Root
  - Leaf
  - Parent / Child
  - Height



Complete Binary Tree

- A complete binary tree
  - Fills every level except possibly the last
  - Fills the last level from left to right
- Height = $O(\log n)$



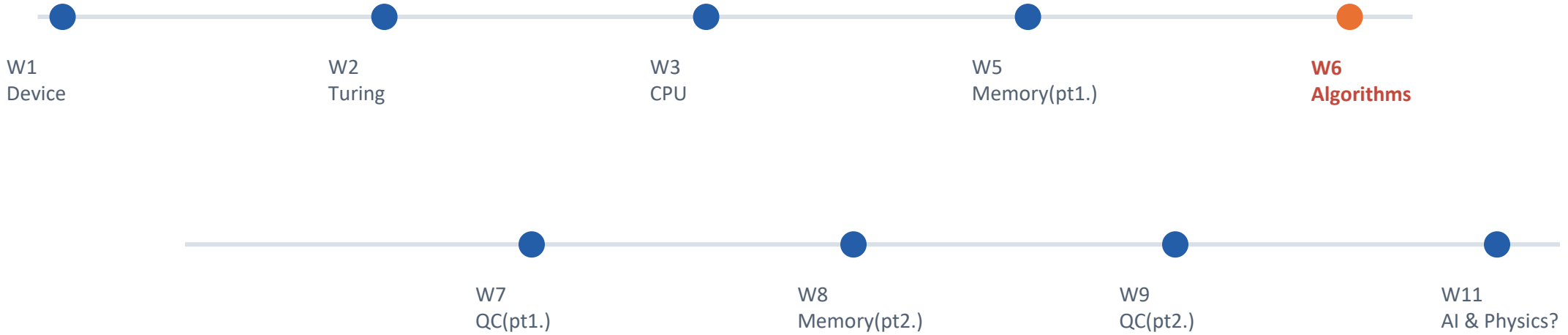
Heap and Priority Queue

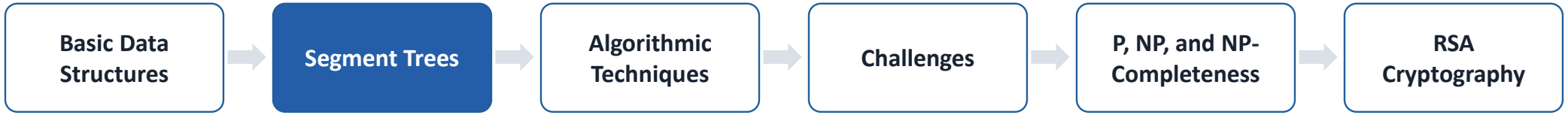
- A binary heap is a complete binary tree satisfying the heap property.
  - Min-Heap : $parent \leq children$
  - Max-Heap : $parent \geq children$
- Main operations
  - push(x) — insert an element
  - pop() — remove the highest-priority element
  - top() — inspect the highest-priority element
- push & pop : $O(\log n)$, top : $O(1)$

Why Heap Operations Are $O(\log n)$

- Insertion
  - Insert at the next available position.
  - Move upward while the heap property is violated.
- Deletion
  - Remove the root
  - Move the last element to the root
  - Move downward until the heap property is restored
- Both follow at most one root-to-leaf path, $O(\log n)$

RoadMap


W1
Device
W2
Turing
W3
CPU
W5
Memory(pt1.)
W6
Algorithms
W7
QC(pt1.)
W8
Memory(pt2.)
W9
QC(pt2.)
W11
AI & Physics?


Basic Data Structures
Segment Trees
Algorithmic Techniques
Challenges
P, NP, and NP-Completeness
RSA Cryptography

A Different Kind of Query

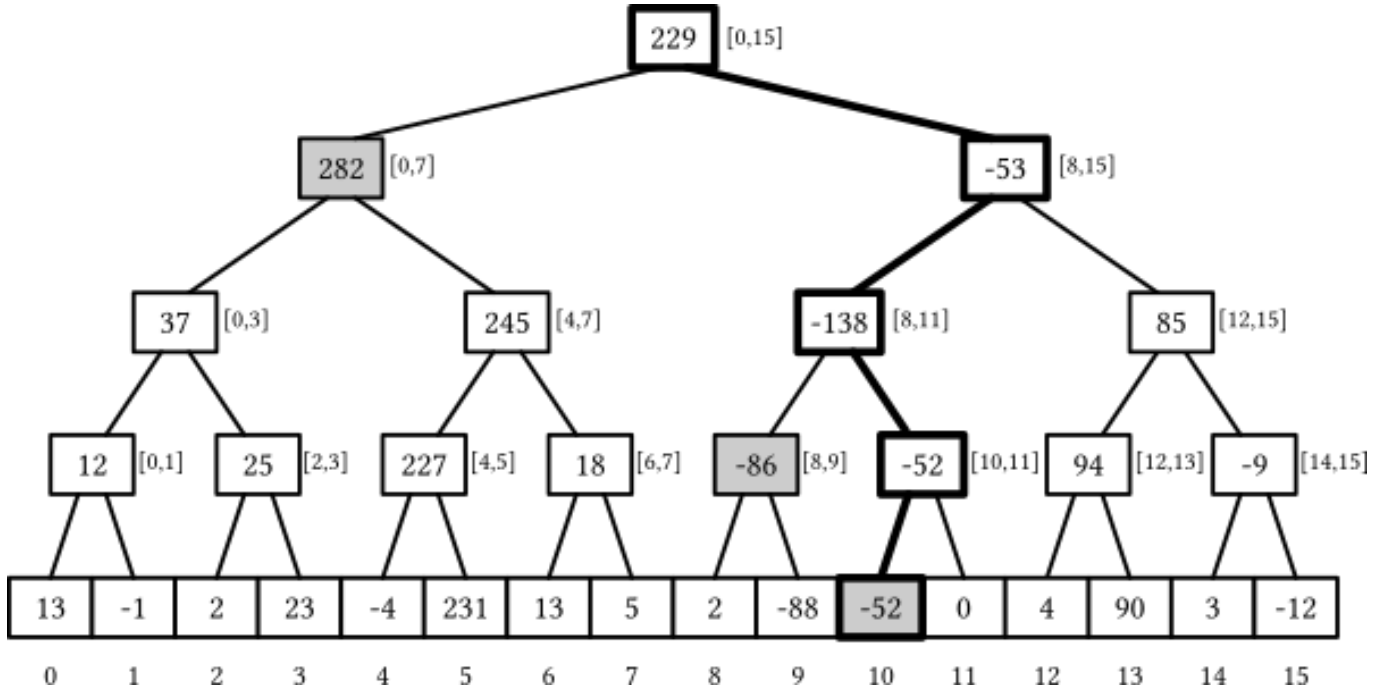
- A Different Problem
  - Given - $a_1, a_2, \cdots, a_n$
  - Range Query - $query(l, r) = a_l + \cdots + a_r$
  - Point Update Query - $update(i, x) : a_i \leftarrow x$

Simple Approaches

- Direct Scan
  - Query : $O(n)$
  - Update : $O(1)$
- Prefix Sum ($P_0 = 0, P_i = P_{i-1} + a_i, sum(l, r) = P_r - P_{l-1}$)
  - Query : $O(1)$
  - Update : $O(n)$
- Can we achieve $O(\log n)$ for both?

Segment Tree Structure

- A segment tree recursively divides an interval into two halves



- Each node stores information about its interval (In this picture : range-sum)

What Can a Segment Tree Store?

- Range Sum
  - $tree[node] = tree[left] + tree[right]$
- Range Maximum
  - $tree[node] = \max(tree[left], tree[right])$
- Range Minimum
  - $tree[node] = \min(tree[left], tree[right])$
- Other examples : Range GCD, Bitwise Operations

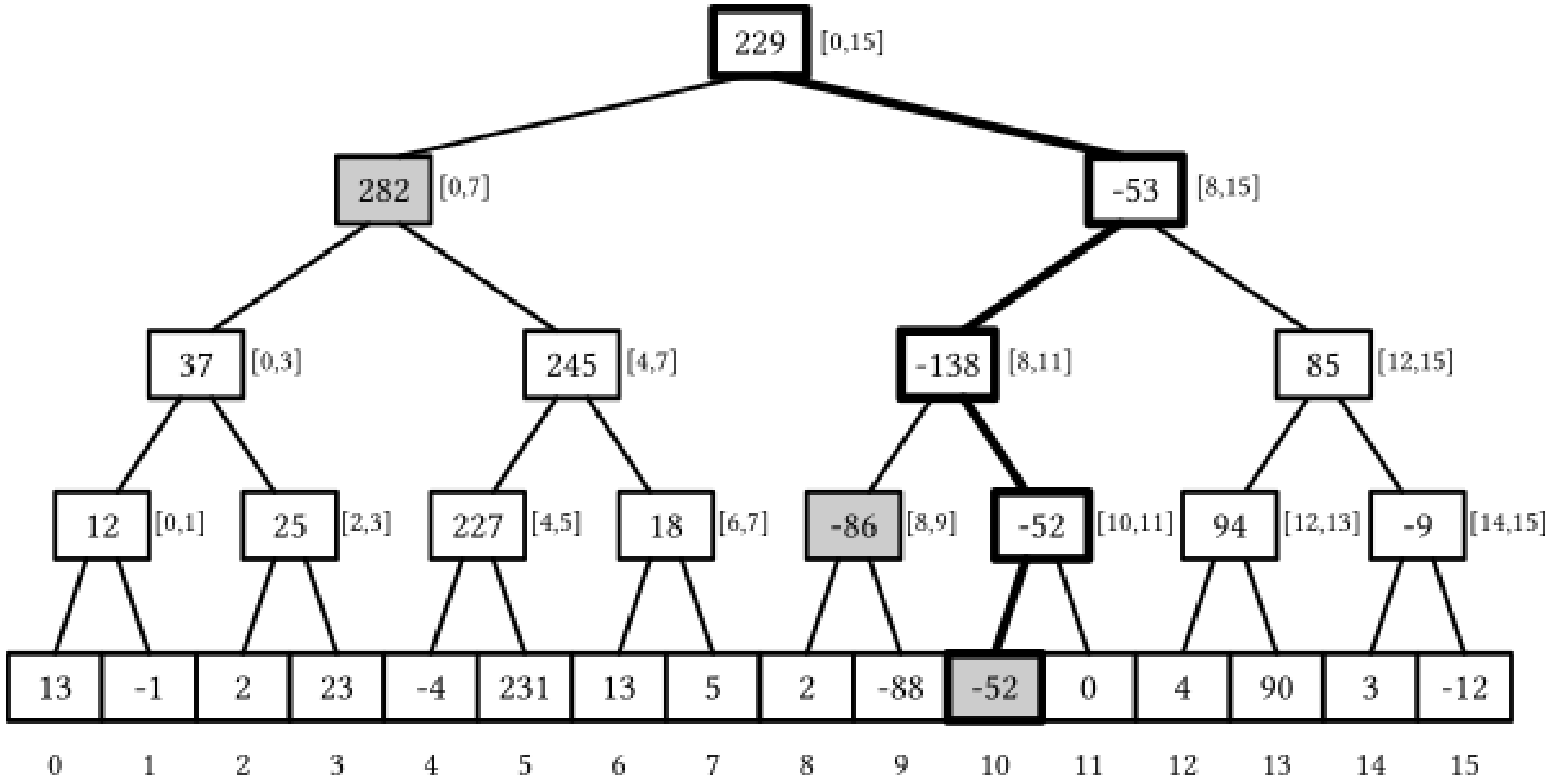
Building a Segment Tree

- Base case
  - $left = right$
  - Store the corresponding array element
- Otherwise
  - Divide the interval into two halves
  - Build both children
  - Combine their values
- Each node is computed once, $O(n)$

Range Query

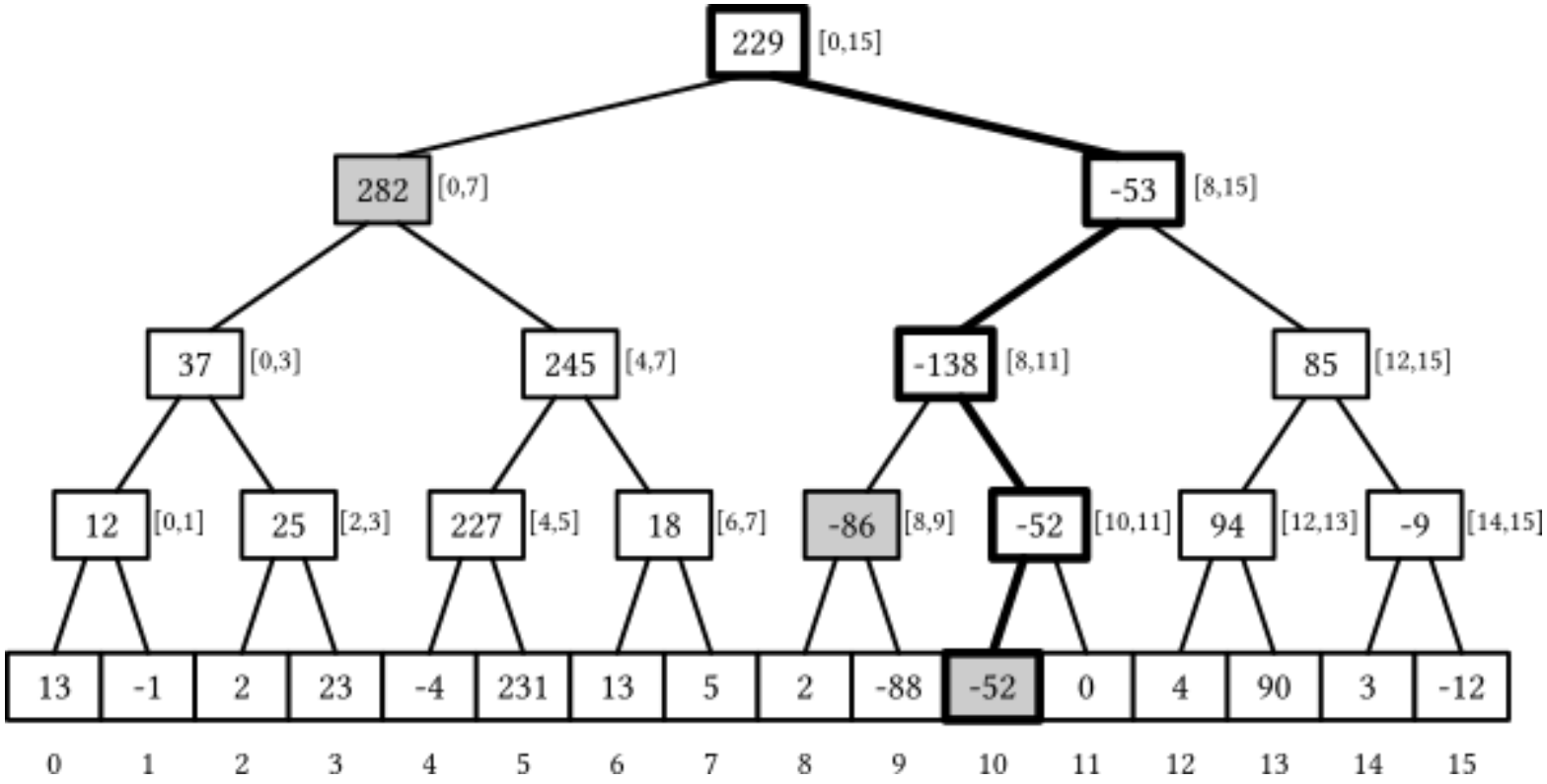
- For node interval $[s, e]$ and query interval $[l, r]$
- No overlap
  - Ignore this node
- Complete overlap
  - Use the stored value directly
- Partial overlap
  - Visit both children and combine their answers

Query Example : $[3, 12]$



Point Update

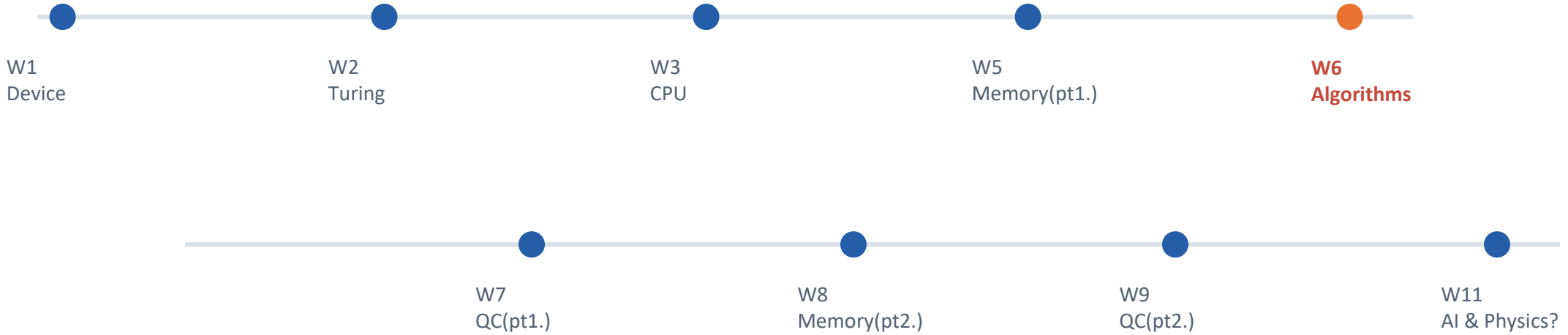
- Suppose : Update a_{10}

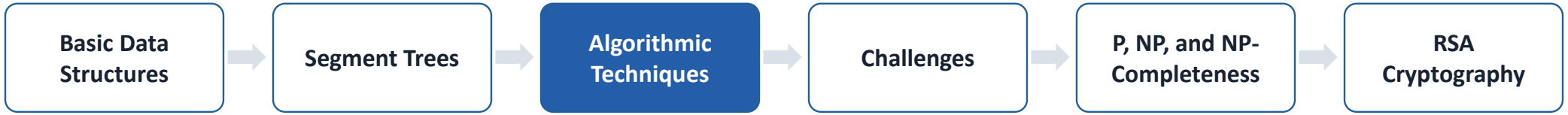


Sorting

- Sorting rearranges elements into a specified order
  - Bubble Sort, Selection Sort, Insertion Sort - $O(n^2)$
  - Heap Sort, Merge Sort, Quick Sort (Worst $O(n^2)$) - $O(n \log n)$
- Sorting is useful for
  - Binary Search
  - Coordinate Compression
  - Revealing order and structure
- $[5, 2, 4, 6, 1, 3] \rightarrow [1, 2, 3, 4, 5, 6]$

RoadMap


W1
Device
W2
Turing
W3
CPU
W5
Memory(pt1.)
W6
Algorithms
W7
QC(pt1.)
W8
Memory(pt2.)
W9
QC(pt2.)
W11
AI & Physics?


Basic Data Structures
Segment Trees
Algorithmic Techniques
Challenges
P, NP, and NP-Completeness
RSA Cryptography

Binary Search

- Binary search repeatedly eliminates half of a sorted search space
- For a sorted array $a_1 \leq a_2 \leq \cdots \leq a_n$
  - each step reduces the remaining search range by approximately half
- $O(\log n)$
- Binary search can find
  - The first value $\geq x$
  - The first value $> x$

Greedy Algorithms

- A greedy algorithm repeatedly makes the choice that appears best right now
- General idea
  - Choose a locally optimal option
  - Commit to the choice
  - Continue with the remaining problem
  - Do not reconsider previous decisions
- $\text{Local Optimum} \overset{?}{\Longrightarrow} \text{Global Optimum}$

Coin Change Problem

- Given a set of coin denominations and a target amount,
  find the minimum number of coins needed to make that amount
- First Case : $C = \{1, 5, 10, 50\}$, $x = 76$
  - Greedy works for this coin system
  - $x = 50 + 10 + 10 + 5 + 1$
- Second Case : $C = \{1, 3, 4\}$, $x = 6$
  - Greedy fails for this coin system
  - $x = 4 + 1 + 1 = 3 + 3$

Why Greedy Needs a Proof

- When Does Greedy Work?
- Greedy-choice property
  - A locally optimal choice can be part of a globally optimal solution
- Optimal substructure
  - After making the choice, the remaining problem also has an optimal solution

Dynamic Programming

- Dynamic programming stores solutions to smaller subproblems and reuses them
- Useful when a problem has
  - Overlapping subproblems
  - Optimal substructure

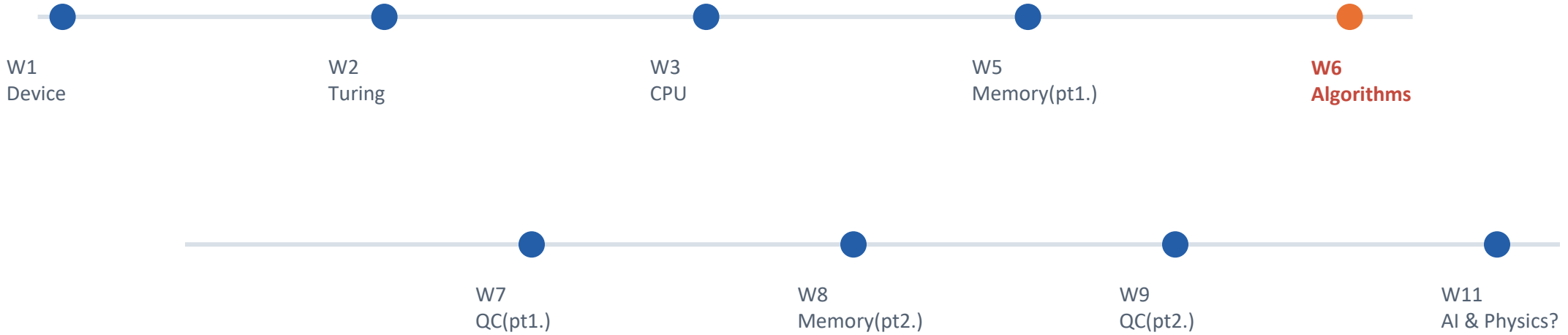
Designing a DP

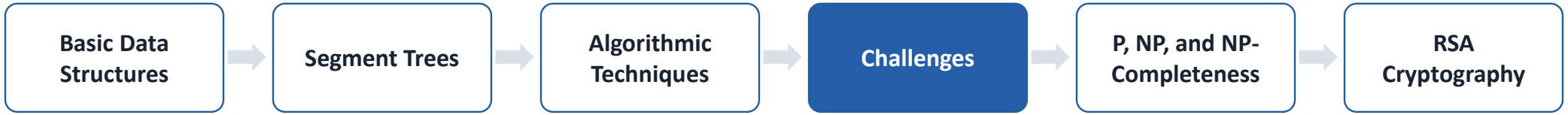
- How to Design a DP
  - Define the state
  - Derive the transition
  - Determine the base cases
  - Decide the evaluation order
  - Identify the final answer
- The most important question : What does $dp[i]$ mean?

Coin Change with DP

- Define $dp[i]$
  - $dp[i] = \text{minimum number of coins needed to make } i$
- Base
  - $dp[0] = 0$
- Transition
  - $dp[i] = \min_{\substack{c \in C \\ c \le i}} (dp[i-c] + 1)$
- Previous Case : $C = \{1, 3, 4\}$, $x = 6$ - $dp[6] = 2$

RoadMap


W1
Device
W2
Turing
W3
CPU
W5
Memory(pt1.)
W6
Algorithms
W7
QC(pt1.)
W8
Memory(pt2.)
W9
QC(pt2.)
W11
AI & Physics?


Basic Data Structures
Segment Trees
Algorithmic Techniques
Challenges
P, NP, and NP-Completeness
RSA Cryptography

Challenge 1

- There are N potions in a line. Drinking potion i changes your health by a_i. For each potion, you may either drink or skip. Your health started from 0 and never become negative. Maximize the number of potions you drink.

- Input
  - Integer N $(1 \leq N \leq 2\times10^5)$
  - $a_1, a_2, \cdots, a_N$ $(-10^9 \leq a_i \leq 10^9)$
- Output
  - Maximum number of potions you can drink

- Example

Input
6 4 -4 1 -3 1 -3
Output
5

Challenge 2

- N people stand in a line. Two people can see each other if every person standing between them is shorter than both of them. Count the number of visible pairs.

- Input
  - Integer N $(1 \leq N \leq 2{\times}10^5)$
  - $h_1, h_2, \cdots, h_N$ $(1 \leq h_i \leq 10^9)$
  - All heights are distinct
- Output
  - Number of visible pairs

- Example

Input
5 4 3 1 5 2
Output
6

Challenge 3

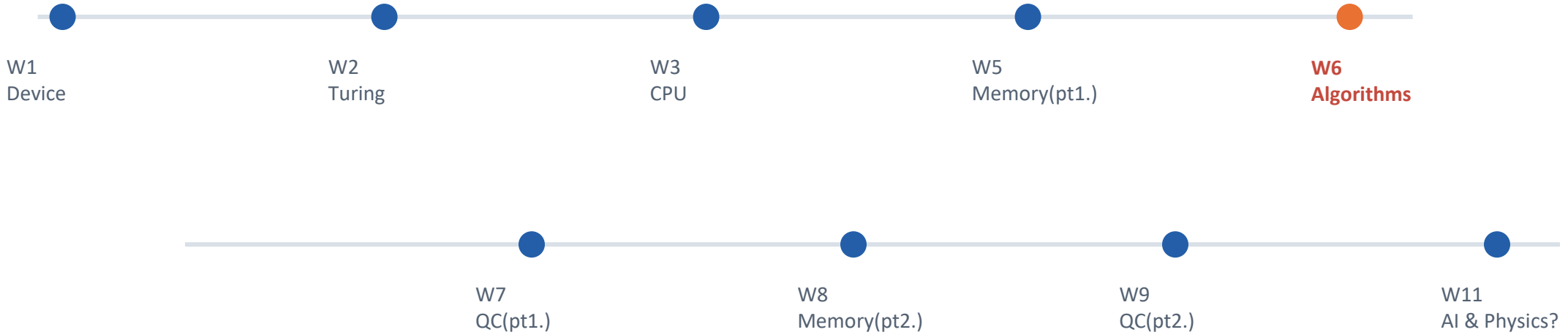
- Given an array $a_1, a_2, \ldots, a_N$, choose a set of non-overlapping segments. For every selected segment $[l, r]$: $a_l + a_{l+1} + \cdots + a_r \geq 0$. Maximize the total length of the selected segments.

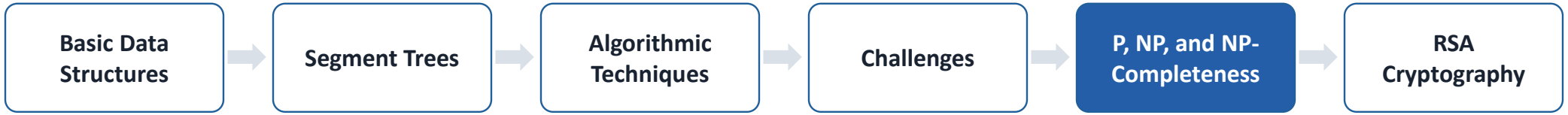
- Input
  - Integer N $(1 \leq N \leq 2\times10^5)$
  - $a_1, a_2, \cdots, a_N$ $(-10^9 \leq a_i \leq 10^9)$
- Output
  - Maximum total length of selected segments

- Example

Input
5 3 -3 -2 5 -4
Output
4

RoadMap


W1
Device
W2
Turing
W3
CPU
W5
Memory(pt1.)
W6
Algorithms
W7
QC(pt1.)
W8
Memory(pt2.)
W9
QC(pt2.)
W11
AI & Physics?


Basic Data Structures
Segment Trees
Algorithmic Techniques
Challenges
P, NP, and NP-Completeness
RSA Cryptography

From Fast Algorithms to Hard Problems

- We improved
  - $O(n^2) \rightarrow O(n \log n)$
- This raises a deeper question
  - Can every computational problem be solved efficiently?

Decision Problems

- A decision problem has two possible answers : Yes / No
- Optimization Coin Change
  - Find the minimum possible number of coins
- Decision Coin Change
  - Is there a way to make exactly x using at most k coins?

P and NP

- P
  - Decision problems solvable in polynomial time : $O(n^k)$ for some constant k
- NP
  - Decision problems whose proposed solutions can be verified in polynomial time
  - $P \subseteq NP$
- NP does not mean “not polynomial”

NP-Hard and NP-Complete

- NP-Hard
  - Every problem in NP can be polynomially reduced to it
- NP-Complete
  - It is in NP
  - It is NP-hard
  - It can be verified efficiently, but it is at least as hard as every problem in NP.
- If any NP-complete problem can be solved in polynomial time
  - $P = NP$

How Do We Find NP-Complete Problems?

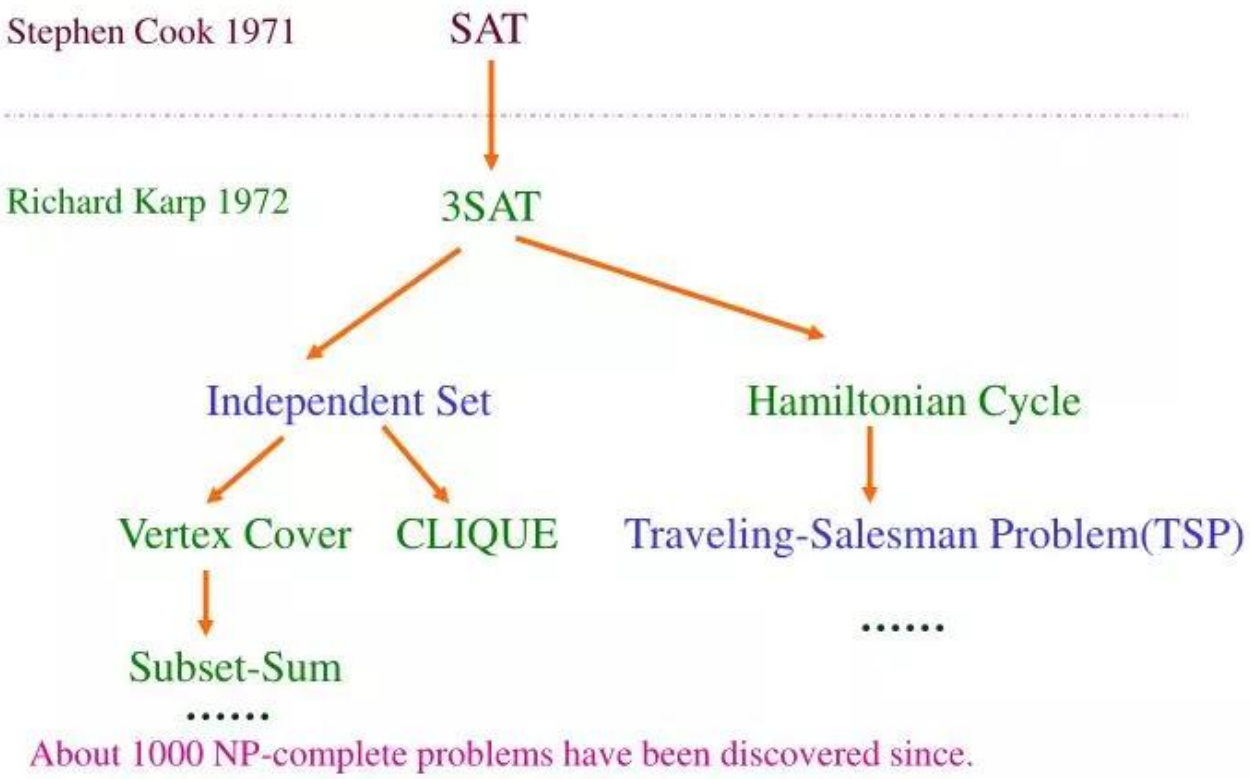
- The key technique : Polynomial time reduction
- Suppose problem A is already known to be NP-complete
- If we can transform every instance of A into an instance of B in polynomial time
  - $A \leq_p B$
- Then solving B efficiently would also solve A efficiently
  - B is at least as hard as A

The Beginning of NP-Completeness

- 1971 - SAT
  - Stephen Cook showed that Boolean Satisfiability (SAT) is NP-complete
  - SAT became the starting point for proving other problems NP-complete
- The general strategy
  - $\text{SAT} \leq_p \text{other problems}$

Building a Chain of Reductions

- Once one NP-complete problem is known, we can build a chain:
  - $SAT \leq_p 3SAT \leq_p \cdots \leq_p Hamiltonian\ Cycle \leq_p TSP$



Hamiltonian Cycle and TSP

- Hamiltonian Cycle
  - Is there a cycle that visits every vertex exactly once and returns to the starting vertex?
  - The Hamiltonian Cycle decision problem is NP-complete
- Traveling Salesman Problem (TSP)
  - Is there a tour that visits every vertex exactly once, returns to the starting vertex, and has a total cost at most K?

From Hamiltonian Cycle to TSP

- Reduction: Hamiltonian Cycle → TSP
- Given a graph $G = (V, E)$ construct a complete weighted graph.
- For every pair of vertices u, v
  - $w(u,v) = \begin{cases} 1 & \text{if } (u,v) \in E \\ 2 & \text{if } (u,v) \notin E \end{cases}$
- Let $n = |V|$
- Ask Is there a tour with total cost at most n?

What Did We Prove?

- We constructed the TSP instance in polynomial time
  - $Hamiltonian\ Cycle \leq_p TSP$
- Hamiltonian Cycle is NP-complete
  - Therefore, $\text{Decision } TSP \text{ is NP}-\text{hard}$
- The decision version of TSP is also in NP
  - $\text{Decision } TSP \text{ is NP}-\text{complete}$

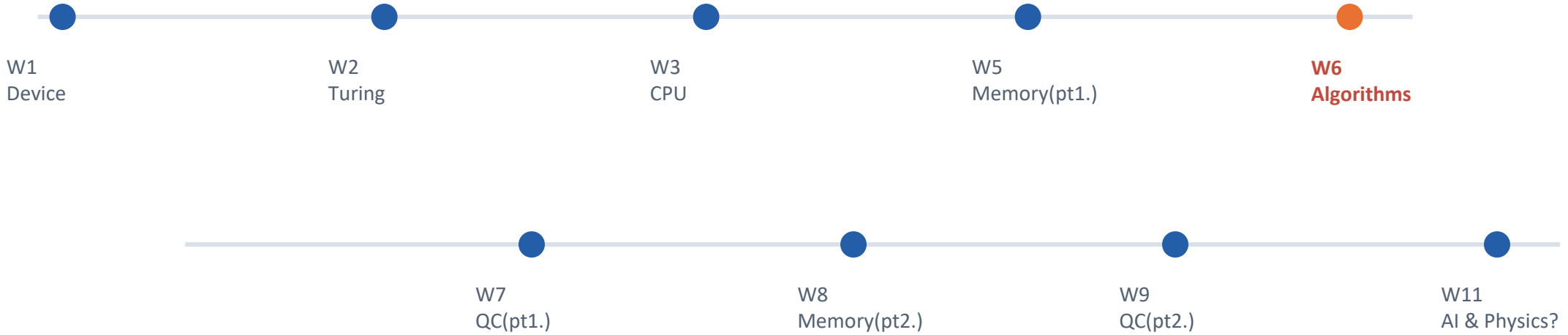
The General Recipe

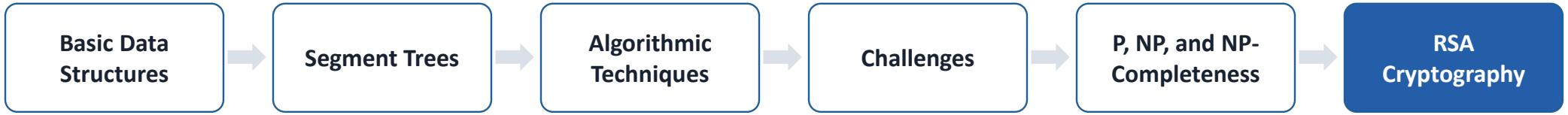
- Show that the problem is in NP
- Choose a known NP-complete problem A
- Construct a polynomial-time reduction : $A \leq_p B$
- Prove : $x \in A \Leftrightarrow f(x) \in B$
- Conclusion : B is $\mathrm{NP-complete}$

Why Are Reductions Useful?

- Suppose a new problem resembles a known NP-complete problem
- Can a known NP-complete problem be encoded inside this problem?
- If yes, this may explain why finding an efficient exact algorithm is difficult

RoadMap


W1
Device
W2
Turing
W3
CPU
W5
Memory(pt1.)
W6
Algorithms
W7
QC(pt1.)
W8
Memory(pt2.)
W9
QC(pt2.)
W11
AI & Physics?


Basic Data Structures
Segment Trees
Algorithmic Techniques
Challenges
P, NP, and NP-Completeness
RSA Cryptography

Public-Key Cryptography

- Public Key : Shared with everyone
- Private Key : Kept secret
- The goal
  - Easy to perform the forward operation, but computationally difficult to reverse it without secret information

RSA Key Generation

- Choose two large primes : p, q
- Compute $n = pq, \phi(n) = (p-1)(q-1)$
- Choose e such that $\gcd(e, \phi(n)) = 1$
- Find d such that $ed \equiv 1 \left(\mathrm{mod}\ \phi(n)\right)$
- Public key : (n, e)
- Private key : d

Encryption and Decryption

- Encryption : $c = m^e \mod n$
- Decryption : $m = c^d \mod n$

Why Is RSA Related to Hardness?

- Easy : $p, q \longrightarrow n = pq$
- Believed difficult for classical computers : $n \longrightarrow p, q$
- Security of RSA is closely related to difficulty of factoring large composite integers
- Integer factorization is not known to be NP-complete
- RSA is not an application of NP-completeness

References

[1] Algorithmica. "Segment Trees." https://en.algorithmica.org/hpc/data-structures/segment-trees/

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. Introduction to Algorithms, 4th ed. MIT Press, 2022.

[3] Codeforces. "1526C2 — Potions (Hard Version)." https://codeforces.com/problemset/problem/1526/C2

[4] Baekjoon Online Judge. "3015 — Oasis Reunion." https://www.acmicpc.net/problem/3015 (Challenge problem adapted with distinct heights.)

[5] Codeforces. "1788E — Sum Over Zero." https://codeforces.com/problemset/problem/1788/E

References

[6] Cook, S. A. "The Complexity of Theorem-Proving Procedures." STOC, pp. 151–158, 1971.

DOI: 10.1145/800157.805047

[7] Karp, R. M. "Reducibility Among Combinatorial Problems." Complexity of Computer Computations, pp. 85–103, 1972.

DOI: 10.1007/978-1-4684-2001-2_9

[8] Rivest, R. L., Shamir, A., and Adleman, L. "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems." Communications of the ACM, 21(2), pp. 120–126, 1978.

DOI: 10.1145/359340.359342